

Automação de testes

Testes automatizados são aqueles realizados com a ajuda de ferramentas de software, que executam os testes automaticamente sem a necessidade de intervenção humana. Eles são projetados para verificar o comportamento de um sistema de forma repetitiva e eficiente, garantindo que o código funcione conforme esperado ao longo do tempo, especialmente quando há mudanças ou novas implementações.

Os testes automatizados envolvem a criação de scripts de teste que simulam a interação do usuário ou verificam o comportamento do código. Esses testes são executados de forma contínua sempre que houver alterações no sistema, podendo ser realizados de maneira frequente e em diferentes versões do software.

Em vez de um testador realizar as verificações manualmente, a ferramenta executa os testes, compara os resultados com os resultados esperados e gera um relatório.

1. Criação de Scripts de Teste

Testes automatizados são baseados em **scripts de teste** (também chamados de **test cases automatizados**) que são escritos em uma linguagem de programação ou usando ferramentas específicas de teste. O script define:

- **Ação:** O que deve ser feito no sistema (ex.: clicar em um botão, preencher um formulário).
- **Expectativa:** O que o sistema deve fazer em resposta (ex.: exibir uma mensagem, redirecionar para uma página).
- **Resultado esperado:** Comparar se o sistema retorna o que era esperado.

2. Execução dos Testes

Depois que os scripts são escritos, os testes são **executados automaticamente** pela ferramenta. O sistema realiza as ações definidas no script e, em seguida, verifica os resultados.

3. Comparação de Resultados

Durante a execução do teste, os resultados são **comparados automaticamente** com os resultados esperados. Se houver divergências (ou seja, o comportamento do sistema não corresponder ao esperado), o teste é considerado **falho**.

4. Relatórios

Após a execução, a ferramenta gera um **relatório** que indica quais testes passaram, que falharam e os motivos para falha, se houver. Esses relatórios são fundamentais para os desenvolvedores corrigirem problemas rapidamente.

Vantagens da Automação de Testes

1. Eficiência e Velocidade

A automação de testes permite **executar testes repetitivos e demorados de forma mais rápida e consistente**. Ao automatizar tarefas como testes de regressão, a equipe pode **liberar recursos** para se concentrar em áreas mais complexas.

2. Redução de Erros Humanos

Ao remover o teste manual, a probabilidade de **erros humanos** é reduzida. Como os testes são executados de forma automática, não há risco de esquecer um caso de teste ou falhar na execução de etapas.

3. Escalabilidade

Os testes automatizados podem ser **executados frequentemente e em várias máquinas simultaneamente** (por exemplo, em ambientes de integração contínua), o que facilita o teste de múltiplas versões e plataformas do software.

4. Reusabilidade

Uma vez que os testes são automatizados, os mesmos scripts podem ser **reutilizados em versões futuras** do software, economizando tempo e esforço ao longo do tempo.

5. Feedback Rápido

Com a automação de testes, as equipes podem obter **feedback rápido** sobre a qualidade do código, o que é essencial para métodos ágeis de desenvolvimento e entrega contínua.

Desvantagens da Automação de Testes

1. Custo Inicial

A criação de **scripts de teste** automatizados pode demandar **tempo e esforço significativos** no início, o que pode ser um investimento considerável, especialmente para equipes pequenas ou sem experiência em automação.

2. Manutenção

A manutenção dos **scripts de teste** é uma tarefa contínua. Cada vez que o código ou a interface do usuário mudar, os testes automatizados também precisam ser ajustados. Se não forem mantidos corretamente, podem **se tornar obsoletos**.

3. Testes de Usabilidade

Testes automatizados são **menos eficazes para avaliar a experiência do usuário** ou problemas de usabilidade, já que não podem captar a **percepção humana** de interatividade e design.

4. Não Detecta Todos os Tipos de Erros

Automatizar testes é eficaz para **erros técnicos** e **funcionais**, mas alguns **problemas lógicos, de design ou complexos** podem não ser detectados de maneira eficaz com scripts automatizados.

Práticas Recomendadas para Automação de Testes

1. Planejamento Cuidadoso

É importante planejar **quais testes devem ser automatizados** com base no impacto, na repetição e no risco, para não desperdiçar recursos em testes de baixo valor.

2. Foco em Testes de Valor

Automatize testes que trarão **maior valor**, como testes críticos, de regressão, de API e de desempenho.

3. Mantenha os Scripts Atualizados

Mantenha os **scripts de teste atualizados** sempre que houver mudanças no código, funcionalidades ou na interface do usuário.

4. Integração Contínua

Integre os testes automatizados em uma **pipeline de integração contínua (CI)**, para que os testes sejam executados automaticamente sempre que houver novas atualizações no código.

Embora a automação de testes traga muitos benefícios, ela também apresenta uma série de **desafios** que podem impactar a eficácia e a eficiência do processo. Compreender esses desafios é crucial para superá-los e garantir o sucesso da automação no desenvolvimento de software.

1. Identificação de Testes para Automação

- Desafio: Quais Testes Automação Devo Priorizar?

A escolha dos **testes certos para automação** é uma das decisões mais difíceis. Nem todos os testes se beneficiam da automação. Testes **altamente**

interativos, de experiência do usuário ou com grande variação em resultados podem ser difíceis de automatizar.

Solução:

- Priorize **testes de regressão, testes funcionais críticos e testes repetitivos**.
- Avalie testes que **não exigem alterações frequentes** na interface ou no fluxo.

2. Complexidade na Criação de Scripts

- **Desafio: Desenvolvimento de Scripts de Teste Complexos**

A criação de **scripts de teste automatizados** pode ser complexa, especialmente para testes que envolvem fluxos complicados, múltiplos sistemas ou integração com outros componentes.

Solução:

- Use ferramentas de **frameworks de automação** (como **Selenium, Cypress, JUnit**) que oferecem recursos avançados de abstração.
- Invista em **treinamento contínuo** da equipe para melhorar a habilidade de escrever scripts eficientes.

3. Manutenção de Testes Automatizados

- **Desafio: Manutenção Contínua dos Scripts**

À medida que o software evolui, a **manutenção dos scripts de teste** se torna um desafio. Mudanças no código, na interface ou na funcionalidade podem **quebrar** os testes automatizados, exigindo revisões constantes.

Solução:

- **Atualize os scripts de teste regularmente** para refletir as mudanças no sistema.
- Estabeleça uma **estratégia de versionamento** para os testes e código-fonte.

4. Mudanças no Sistema e Interdependências

- **Desafio: Alterações Frequentes no Sistema**

Se o sistema sofre **mudanças frequentes** (seja no design, arquitetura ou funcionalidades), isso pode exigir alterações constantes nos **scripts de teste**, tornando a automação menos eficiente.

Solução:

- Adote práticas de **desenvolvimento ágil e integração contínua** para manter a automação em sintonia com as mudanças.
- Use **abstração** no design dos testes para que mudanças no sistema não impactem diretamente os testes.

5. Custo Inicial de Implementação

- **Desafio: Investimento Inicial**

O custo inicial de automação de testes pode ser alto, especialmente quando se trata de **configurar a infraestrutura, treinar a equipe** e criar os primeiros scripts de teste. Isso pode ser um obstáculo, especialmente em equipes menores ou com orçamento limitado.

Solução:

- Comece com **testes pequenos e essenciais**, e expanda gradualmente a automação.
- **Use ferramentas de código aberto** para reduzir os custos com licenciamento de ferramentas caras.

6. Testes de Ambientes Complexos

- Desafio: Configuração de Ambientes de Teste

Ambientes de teste que envolvem várias plataformas, integrações com sistemas externos ou servidores podem ser difíceis de configurar e manter. Testar em **diversos navegadores, sistemas operacionais ou versões de dispositivos** aumenta a complexidade.

Solução:

- Utilize **containers Docker** ou **ambientes virtuais** para simular diferentes configurações.
- **Automatize a configuração do ambiente de teste** sempre que possível.

7. Falta de Estabilidade nas Ferramentas de Automação

- Desafio: Instabilidade das Ferramentas de Teste

As ferramentas de automação podem **ficar desatualizadas** ou ter problemas de compatibilidade com novas versões do software, ou tecnologias. Isso pode levar a **erros imprevistos e interrupções nos testes**.

Solução:

- Mantenha as ferramentas de automação **atualizadas**.
- Faça uma **pesquisa cuidadosa** antes de escolher uma ferramenta de automação, para garantir sua **estabilidade** e **suporte a longo prazo**.

8. Falta de Feedback Imediato e Visualização

- Desafio: Dificuldade em Obter Feedback Rápido

A automação de testes nem sempre oferece **feedback imediato** ou **visualizações claras** sobre falhas, especialmente em testes complexos. Sem uma **interface amigável** para resultados, a equipe pode gastar mais tempo analisando os logs.

Solução:

- Use ferramentas que forneçam **relatórios visuais** e **logs detalhados** para fácil

análise.

- **Integre testes automatizados** em sistemas de **integração contínua (CI)**, para obter feedback em tempo real.

9. Desafios de Conhecimento e Habilidades da Equipe

- **Desafio: Falta de Conhecimento em Automação**

A equipe de desenvolvimento pode não ter experiência ou conhecimento suficiente sobre ferramentas de automação, ou a **escrita de scripts de teste**, o que pode prejudicar a implementação da automação.

Solução:

- **Treinamento e capacitação** contínuos para desenvolvedores e testadores.
- **Contratação de especialistas** ou consultores para auxiliar na configuração inicial e nas melhores práticas.

10. Testes de Usabilidade e Experiência do Usuário

- **Desafio: Limitações da Automação para Testes de Usabilidade**

Automatizar testes de **experiência do usuário (UX)** ou de **design** é um desafio, já que a automação não pode avaliar fatores subjetivos como **intuitividade**, **estética** ou **facilidade de uso** de um sistema.

Solução:

- Use a automação para testar aspectos **funcionais** e **de desempenho**, mas mantenha os testes **de usabilidade** manuais, com base em feedback humano.
- A automação de testes é facilitada por uma variedade de ferramentas especializadas, que são usadas para testar diferentes tipos de aplicações, como **aplicações web**, **móveis**, **APIs** e **sistemas de integração contínua**. Aqui estão algumas das **principais ferramentas** para automação de testes, divididas por tipo de teste e finalidade.

1. Ferramentas para Testes de Aplicações Web

· **Selenium**

- **O que é:** Uma das ferramentas mais populares para **testes de interface web**. O Selenium permite a automação de testes para aplicativos web em vários navegadores.
- **Recursos:** Suporta **Java**, **C#**, **Python**, entre outras linguagens. Pode ser integrado a ferramentas de CI/CD e é muito utilizado para **testes de regressão** e **automação de navegação**.

· **Cypress**

- **O que é:** Uma ferramenta moderna para testes de frontend em aplicações web.

- **Recursos:** Fácil de usar, com suporte para JavaScript. Permite testes de integração e unitários, e é muito usado para testes de UI interativos. Ideal para ambientes JavaScript/React.

- **Playwright**

- **O que é:** Semelhante ao Selenium, mas mais rápido e mais recente. Suporta múltiplos navegadores, incluindo Chrome, Firefox e WebKit.

- **Recursos:** Permite testes de aplicações web e mobile web, e permite a captura de vídeos e screenshots durante os testes.

- **TestComplete**

- **O que é:** Uma ferramenta de testes funcionais e de regressão para aplicações web.

- **Recursos:** Suporta testes baseados em script e testes sem script, além de permitir a gravação e execução automática de testes.

2. Ferramentas para Testes de Aplicações Móveis

- **Appium**

- **O que é:** Ferramenta de automação de testes móveis para Android e iOS.

- **Recursos:** Suporta várias linguagens como Java, JavaScript, Python e Ruby. É útil para testes de integração, funcionais e de regressão em aplicativos móveis nativos e híbridos.

- **Espresso**

- **O que é:** Ferramenta de testes de interface para Android, criada pelo Google.

- **Recursos:** Rápido e confiável, é ideal para testes de UI em aplicações Android. Usado principalmente para testes de regressão e verificação de componentes UI.

- **XCUITest**

- **O que é:** A ferramenta de testes de UI da Apple para iOS.

- **Recursos:** Integrado ao Xcode, o XCUITest é eficaz para testar funcionalidade de aplicativos iOS, incluindo testes de usabilidade e UI.

3. Ferramentas para Testes de APIs

- **Postman**

- **O que é:** Ferramenta popular para testes de APIs RESTful.

- **Recursos:** Permite testes manuais e automatizados de APIs, com uma interface gráfica intuitiva. Suporta scripts de pré-requisição e testes pós-resposta.

- **RestAssured**

- **O que é:** Framework para testes de APIs RESTful, escrito em Java.
- **Recursos:** Muito usado em testes de integração e validação de respostas de APIs. Oferece suporte a testes de autenticação e autorização.

· SoapUI

- **O que é:** Ferramenta robusta para testes de APIs SOAP e REST.
- **Recursos:** Suporta testes de carga e testes de segurança, além de testes funcionais de serviços web.

4. Ferramentas para Testes de Desempenho

· Apache JMeter

- **O que é:** Ferramenta de testes de carga e performance amplamente usada.
- **Recursos:** Suporta testes de desempenho para servidores web, APIs, bases de dados e aplicações móveis. Ideal para testes de estresse, de carga e de tempo de resposta.

· Gatling

- **O que é:** Ferramenta de testes de carga para aplicações web e APIs.
- **Recursos:** Utiliza Scala e tem suporte para testes em alta escala. Permite a criação de cenários complexos de load testing.

· LoadRunner

- **O que é:** Ferramenta da Micro Focus para testes de performance.
- **Recursos:** Suporta testes de carga, stress e performance em diversas plataformas e aplicativos. Pode simular milhares de usuários simultâneos.

5. Ferramentas de Integração Contínua (CI/CD)

· Jenkins

- **O que é:** Ferramenta de integração contínua e entrega contínua amplamente usada.
- **Recursos:** Automatiza a execução de testes automatizados, a compilação do código e a implantação de builds. Pode ser integrado a várias ferramentas de testes e monitoramento.

· CircleCI

- **O que é:** Ferramenta de integração contínua e entrega contínua.
- **Recursos:** Oferece execução de testes automáticos durante o ciclo de integração, com uma configuração de pipeline simples e eficiente.

· GitLab CI/CD

- **O que é:** Ferramenta de CI/CD integrada ao GitLab.
- **Recursos:** Permite a automação de pipelines, execução de testes e gerenciamento de versões diretamente dentro do repositório de código.

6. Ferramentas para Testes de Acessibilidade

· axe-core

- **O que é:** Ferramenta de teste de acessibilidade para web.
- **Recursos:** Detecta problemas de acessibilidade em websites e oferece integrações com várias ferramentas de automação.

· Wave

- **O que é:** Ferramenta para análise de acessibilidade de páginas web.
- **Recursos:** Realiza testes de conformidade com WCAG (Diretrizes de Acessibilidade para Conteúdo Web).

7. Ferramentas para Testes de Segurança

· OWASP ZAP (Zed Attack Proxy)

- **O que é:** Ferramenta de testes de segurança que permite encontrar vulnerabilidades em aplicações web.
- **Recursos:** Realiza testes de penetração automatizados para detectar falhas de segurança em tempo real.

· Burp Suite

- **O que é:** Ferramenta de segurança que permite realizar testes de penetração em aplicativos web.
- **Recursos:** Oferece um conjunto de ferramentas de auditoria de segurança e é usada para detectar vulnerabilidades críticas.

8. Ferramentas para Testes de Interface de Usuário (UI)

· Sikuli

- **O que é:** Ferramenta de automação baseada em imagens que permite capturar e interagir com a interface de usuário de aplicativos.
- **Recursos:** Útil para testes de UI de aplicações desktop e web, onde as interações são baseadas em elementos visuais.

· Ranorex

- **O que é:** Ferramenta de testes automatizados de UI para desktop, mobile e web.
- **Recursos:** Suporta gravação e execução de testes, além de integrar com ferramentas de CI/CD e fornecer relatórios detalhados.

GitHub da Disciplina

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com->

Conteúdo Bônus

Um bom documentário para assistir relacionado a criação do Linux se chama “The Code: Story of Linux” que mostra a colaboração em larga escala e a importância de testes; uma série documental curta chamada “Agile Testing Days Docuseries” que são entrevistas e painéis com engenheiros de teste, abordando automação, TDD e cultura de qualidade em times ágeis; além dessa série podemos citar também um mini-documentário chamado “The Art of Test Automation” que mostra diversos eventos de tecnologia que disponibiliza palestras gravadas com especialistas que abordam casos reais com ferramentas e filosofias da automação de testes.

Referências Bibliográficas

MYERS, Glenford J.; SANDLER, Corey; BADGETT, Tom. The Art of Software Testing. New Jersey: John Wiley & Sons, 2011.

KOSKELA, Lasse. Test Driven: TDD and Acceptance TDD for Java Developers. Greenwich: Manning Publications, 2007.

10 ferramentas para teste automatizado de software. Malu, 16 de janeiro de 2025. Disponível em <https://uds.com.br/blog/ferramentas-para-teste-automatizado-de-software/>. Acesso em 24 de abril de 2025.

Testes automatizados x qualidade de software. Alexandro Lima Santos, 27 de julho de 2024. Disponível em <https://cwi.com.br/blog/testes-automatizados-x-qualidade-de-software/>. Acesso em 24 de abril de 2025.

Conheça 5 ferramentas de automação de testes de software. Voidr, 08 de fevereiro de 2024. Disponível em <https://voidr.co/blog/5-ferramentas-de-automacao-de-testes-de-software/>. Acesso em 24 de abril de 2025.